

Daffodil International University
Department of Software Engineering

Project Proposal

TrustBazaar: A Verified Peer-to-Peer Auction
and Marketplace Platform for Used Goods

Submitted by

S. M. Nihal Ahmed	ID: 232-35-002
Afrim Hossen Khan	ID: 232-35-076
Sabikun Nahar Sinthia	ID: 232-35-475

BSc in Software Engineering
Daffodil International University (DIU)

Submitted for: Capstone Project Approval

September 26, 2026

Contents

1	Motivation	2
1.1	Why Existing Platforms Don't Solve This	2
2	Problem Statement	2
3	Objectives	3
3.1	Aim	3
3.2	Specific Objectives	3
4	Scope	3
4.1	What the Project Covers	3
4.2	What It Deliberately Excludes	3
5	Why This Is a Non-Trivial Build	4
6	Methodology	4
6.1	Approach	4
6.2	Planned Increments	4
6.3	Verification	5
7	Tools and Technology Choices	5
8	Team and Resources	5
9	Timeline (12 Weeks / 3 Months)	5
10	Risks	6
11	Evaluation Criteria	6
12	Expected Outcome	7

1 Motivation

Second-hand goods change hands constantly in Bangladesh, but almost entirely through un-moderated Facebook groups and general classifieds boards such as Bikroy. None of these venues solve the problem that actually makes resale between strangers hard: nobody vouches for either side, nobody holds the money until the item is confirmed delivered, and if a seller wants to run a genuine auction rather than a fixed price, there is no mechanism to stop last-second sniping, fake bidding, or a buyer who simply vanishes after “winning.” Delivery is negotiated ad hoc over chat, with no fare transparency and no fallback if the courier or the buyer disappears.

TrustBazaar is a proposal to close that gap for one well-defined slice of the market: individual-to-individual resale of used goods. Rather than trying to be a general marketplace, the project deliberately narrows itself to four problems that are genuinely hard to get right – identity trust, fair auctions, protected payment, and delivery coordination – and treats them as one connected system instead of four separate features bolted together after the fact.

1.1 Why Existing Platforms Don’t Solve This

It is worth being specific about why the obvious alternatives fall short, since the case for building something new rests on it:

- **Facebook Marketplace / classifieds groups** have no identity verification beyond a social login, no payment handling at all, and no auction mechanic – “highest offer in the comments” has no enforced end time and no record of who actually committed to buy.
- **Bikroy and similar classifieds boards** are listing directories, not transaction platforms. They connect a buyer and seller and then step out of the way; whatever happens with payment and delivery happens entirely off-platform, with no escrow and no dispute mechanism.
- **Daraz and other e-commerce marketplaces** do offer payment and delivery infrastructure, but they are built for businesses selling new inventory at scale, not for one individual selling one used item to another. There is no auction format, and the seller-verification bar is built around business registration, not personal KYC.

None of these platforms combine identity verification, escrow, fair auctions, and delivery coordination into a single flow for individual resale – which is precisely the space TrustBazaar is meant to occupy.

2 Problem Statement

A buyer and seller who have never met are being asked to trust each other with real money and a physical item, with no institution standing behind either promise. Three consequences follow directly from that, and each is a concrete reason informal channels fail in practice:

1. **No accountability.** Anyone can create a profile and list an item; nothing ties a listing to a real, verifiable person, so a defrauded buyer has no recourse and a scammer can simply reappear under a new profile.
2. **No payment safety.** Money changes hands either before or after delivery, with no neutral holding point in between, so either the buyer risks paying for an item that never arrives, or the seller risks shipping an item that is never paid for.
3. **No fair price discovery.** Where sellers do attempt informal “highest offer wins” auctions in comment threads, there is no enforced end time, no protection against a bid placed in the final second, and no verifiable record of who actually won – so the seller has no way to hold the winning bidder to the deal.

Delivery adds a fourth, quieter problem: without a system-computed price, courier cost becomes

another point of dispute layered on top of an already low-trust transaction, and there is no accepted authority to say whether a quoted fare is reasonable.

3 Objectives

3.1 Aim

To design and build a web-based marketplace in which identity verification, escrow-held payment, a fair time-boxed auction mechanism, and delivery coordination work as parts of one settlement flow, for the specific case of individual resale of used goods in Bangladesh.

3.2 Specific Objectives

1. Gate the ability to list or transact behind a KYC check, so every seller and buyer in a completed deal is an identity-verified individual, not an anonymous profile.
2. Build an auction mechanism whose end time and highest bid are decided by the server rather than the client, so a fast connection or a manipulated timestamp cannot win an auction unfairly, and add an automatic short extension when a bid arrives near the scheduled close, so a genuine last-second sniper cannot win by timing alone.
3. Route every payment – mobile financial service or card – through a single escrow step that only releases funds to the seller once the buyer confirms receipt, or automatically after a fixed timeout if the buyer does nothing.
4. Compute the distance between buyer and seller automatically and use it to either suggest a safe public meetup point or book a courier at a fare the system, not either party, has quoted – removing fare negotiation as a point of friction.
5. Keep a readable, auditable record of how each payment was split between seller, courier, and platform, so that a dispute can be resolved by inspecting data rather than by taking someone's word for it.

4 Scope

4.1 What the Project Covers

The system is a single responsive web application (no native mobile app) built around one transaction lifecycle: a user verifies their identity, lists or finds an item, buys it outright or wins it at auction, pays into escrow, receives it via self-pickup or courier, and confirms receipt so funds are released. Chat, ratings, dispute handling, and notifications exist to support that lifecycle – they are supporting infrastructure, not independent features built for their own sake.

4.2 What It Deliberately Excludes

The project is not attempting to be a general e-commerce storefront for new or branded retail, and it is not building a back-office management system. The one internal-facing role in the system, used for KYC review and dispute adjudication, is kept deliberately narrow – giving it general create/edit/delete access over listings, users, or orders would turn the project into exactly the kind of generic admin panel it is trying to avoid being. Real financial transactions and production courier bookings are also out of scope for the academic deliverable; sandbox and test-mode integrations stand in for them, and that substitution is disclosed rather than hidden behind a convincing-looking demo.

5 Why This Is a Non-Trivial Build

A project like this earns its place as a capstone specifically because its interesting parts cannot be faked with a simple database and a form:

- **Concurrency has to actually be handled, not assumed away.** Two people bidding on the same auction in the same second, or two buyers trying to buy the last unit of a fixed-price listing at the same moment, are ordinary situations the system has to get right every single time, not edge cases to patch later once someone complains.
- **Multiple external payment and delivery providers have to agree with each other.** A bKash payment, a Nagad payment, and a card payment all have to collapse into the same internal notion of “this order is paid,” despite arriving through three different APIs with three different confirmation mechanisms; a courier’s fare quote has to reconcile with what the platform actually charged the buyer.
- **Trust decisions have consequences that compound.** A KYC approval, a bid, and a payment release are not independent actions – each one changes what the system is allowed to let happen next, which is a materially different design problem from a typical CRUD workflow where most actions are reversible and independent of each other.

6 Methodology

6.1 Approach

The build proceeds as a sequence of working, demonstrable increments rather than a single delivery at the end. Each increment adds one complete slice of the transaction lifecycle – data model, backend logic, and the corresponding UI together – so that at every stage there is something that can actually be clicked through, not just a partially wired backend sitting behind a placeholder screen.

6.2 Planned Increments

1. **Identity and accounts:** registration, login, and the KYC submission-and-review path that gates everything downstream.
2. **Listings:** creating a fixed-price or auction listing, and browsing/searching/filtering the catalog by category, price, condition, and distance.
3. **The auction path:** server-authoritative bidding, automatic proxy bids, reserve price, and the anti-sniping extension, tested specifically under simultaneous bidders rather than only one bidder at a time.
4. **The fixed-price path:** purchase with a stock decrement that stays correct under simultaneous buyers attempting to claim the last unit.
5. **Payment and escrow:** sandbox integration with the mobile financial services and card processor, and the hold-then-release settlement logic that sits behind both.
6. **Delivery:** distance computation and the self-pickup/courier decision, with a courier sandbox or its documented mock substitute.
7. **Trust and communication layer:** chat, ratings, disputes, and notifications, wired to the specific transactions they support rather than built as generic messaging.
8. **Monetization and hardening:** commission and delivery-margin calculation on top of the existing ledger, followed by a pass on rate-limiting, audit logging, and behavior under concurrent load.

6.3 Verification

Because the riskiest parts of the system are the concurrent paths, automated tests specifically target simultaneous bidding, simultaneous stock decrement, repeated or duplicate payment-webhook delivery, and the correctness of the fee-split arithmetic, rather than only exercising the happy path once. These run automatically on every push to the main branch, so a regression in one of these areas is caught before it ever reaches a demonstration.

7 Tools and Technology Choices

The technology choices below are made for reasons specific to this project, not chosen by default:

- **PostgreSQL** is chosen over a document store because settlement and bidding correctness genuinely depend on transactional (ACID) guarantees – this is a case where that property is load-bearing, not a formality that could be swapped out later.
- **Redis** holds auction countdown state and coordinates bid locks outside of any single server process’s memory, so restarting or scaling the backend cannot accidentally reset or extend a live auction’s true end time.
- **WebSockets / Server-Sent Events** carry live bid updates and chat, since a marketplace where the displayed highest bid is stale by several seconds undermines the fairness the auction engine is meant to guarantee in the first place.
- **Docker Compose** keeps the backend, database, cache, and real-time service as separate containers, which mirrors how the pieces would actually be deployed and keeps local development close to that reality rather than diverging from it.
- Payment and courier integrations use each provider’s sandbox or test mode; where no public courier sandbox exists, a mock service is used and clearly labeled as such rather than presented as a live integration.

8 Team and Resources

The project is undertaken by a three-member team:

- **S. M. Nihal Ahmed** (ID: 232-35-002)
- **Afrim Hossen Khan** (ID: 232-35-076)
- **Sabikun Nahar Sinthia** (ID: 232-35-475)

Having three people changes the methodology in one important way: instead of the strictly sequential, one-slice-at-a-time build described for a solo effort, the workstreams below are split across team members and run partly in parallel once the shared foundation (data model and authentication) is in place, which is what makes a three-month timeline realistic rather than optimistic. A shared Git repository with feature branches and a lightweight code-review step before merging to main keeps the three workstreams from silently drifting apart. Development, sandbox accounts, and hosting are expected to run on free or student-tier offerings (payment provider sandboxes, a free-tier cloud instance or local Docker Compose setup for demonstration), keeping the resource footprint of the build close to zero cost.

9 Timeline (12 Weeks / 3 Months)

The three core workstreams – the auction/listings path, the payment/escrow path, and the delivery/trust-layer path – are assigned to the three team members and developed in parallel from Week 3 onward, against the shared data model and authentication built together in Weeks 1–2.

Weeks	Focus	Owner(s)
1–2	Shared setup: repo, Docker Compose, data model, auth, KYC flow	All three, together
3–5	Auction engine, fixed-price purchase, concurrency tests	Member A
3–5	Payment/escrow integration, settlement ledger	Member B
3–5	Listings, search/discovery, delivery decision engine	Member C
6–7	Integrate the three workstreams into one order lifecycle	All three, together
8	Chat and notifications	Member A
8	Ratings and disputes	Member B
8	Monetization features (commission, delivery margin)	Member C
9	Security hardening: rate-limiting, audit logging, IDOR checks	All three, together
10	End-to-end and load testing of concurrency-sensitive paths	All three, together
11	Bug fixing, System Design Document, README	All three, together
12	Final polish, demonstration rehearsal, submission	All three, together

Member assignment (A/B/C) is deliberately left as a role rather than pinned to a specific name here, so the team can assign it based on who ends up strongest in each area during Weeks 1–2. The auction engine and the payment/escrow flow remain the critical path, since chat, ratings, disputes, and monetization all depend on an order existing and having been paid for correctly – so those two workstreams get first claim on time if Weeks 3–5 run long. If the schedule slips, the features scoped down first are the Should-Have and Could-Have ones (delivery automation, disputes, notifications, featured listings, the Pro Seller tier), since the Must-Have core – KYC, the auction engine, escrow settlement, and chat – is what the project’s central claim depends on.

10 Risks

- **Sandbox access delays.** bKash/Nagad developer sandbox approval can be slow to obtain; if it is not available in time, Stripe test mode alone will be used and the limitation stated openly in the final documentation rather than worked around silently.
- **No public courier sandbox.** Handled by substituting a mock courier service with realistic fare and status behavior, documented explicitly as a substitution rather than presented as if it were live.
- **Concurrency defects.** The single biggest technical risk, mitigated by row-level locking in PostgreSQL and by writing the concurrency tests before the corresponding feature is considered finished, not after a bug is reported.

11 Evaluation Criteria

The project’s success is judged less by feature count and more by whether the core trust guarantees actually hold under stress. Concretely, that means: two simultaneous bids on the same auction never both register as the winning bid; two simultaneous buyers can never both purchase the last unit of a fixed-price listing; an escrow payment is never released to a seller without either explicit buyer confirmation or a genuine timeout; and a payment-webhook delivered twice by a provider never results in the order being charged or settled twice. These four properties, verified by the automated test suite described in the methodology, are treated as the real deliverable – the surrounding features exist to give them a context to operate in.

12 Expected Outcome

By the end of the project, a demonstrable web application should exist in which a verified user can list an item, another verified user can either buy it or win it in a fair, server-timed auction, pay for it safely, receive it through a system-arranged method, and trust that the money only moved when it was supposed to. The accompanying repository, System Design Document, and test suite should make the reasoning behind each of those guarantees inspectable, not just the working demo itself – so that the project stands up to the question “why should anyone trust this,” rather than only to the question “does it work in the demo.”